StageSight: A River Camera That Checks Its Gauge

OpenCV 5 reads USGS staff plates; Claude on AWS decides which instrument to trust

Muhammad Waqar Saleem
Department of Civil and Environmental Engineering, University of Missouri, Columbia, MO, USA
msx9t@missouri.edu ORCID 0009-0004-4976-6979
OpenCV AI Competition 2026, powered by AWS

***Abstract*—A river gauge reports stage from a single pressure sensor. When that sensor drifts, freezes or goes silent, often during the very flood it is meant to record, the record shows no sign of it. Many USGS gauges already carry a camera aimed at a staff plate, but its images are inspected by people, not measured. StageSight turns that camera into an independent second instrument. Classical OpenCV 5 operations and the OpenCV DNN text recognizer read the stage from the plate's printed marks and numbers, *without any gauge data*. On the 29 July 2026 flood at Hancock, NY (USGS 01421500; NWS minor flood, 18.08 ft peak), the camera agrees with the USGS gauge to 0.26 ft RMSE (95% bootstrap interval 0.20–0.31 ft; 29 of 30 readings within 0.5 ft), whereas a regression trained on a smaller September high water has 6.3 ft RMSE. An event-gated agent (Claude on Amazon Bedrock) compares camera and gauge, gathers evidence with tools, and queues sensor flags and flood alerts for human approval. In three replays of each injected fault, Claude Opus 4.6 caught all four gauge faults and blamed the camera in both camera-fault cases in every run, where a rule baseline blamed the healthy gauge; it also flagged the healthy camera during a frozen gauge, and Claude Sonnet 4.6 missed a slow gauge drift in two of three runs. The system runs live on AWS Lambda (Graviton), EventBridge, S3, SNS and Bedrock, and unchanged on a laptop CPU; on Graviton4 the Cloud-Optimized OpenCV build (COOL) gives output identical to stock OpenCV.**

I. Introduction

Stage, the water level at a gauge, drives flood forecasts, warnings and discharge records. A gauge measures it with one sensor, here a pressure transducer, and that sensor fails in quiet ways: it fouls with sediment, its line freezes, its telemetry drops out. Hydrographers find these faults later, during record review; forecasters and emergency managers may act on the bad value in real time.

The USGS operates cameras at many of its gauges and serves their images through the NIMS API [1]. Published camera methods measure stage from such images with deep segmentation and photogrammetry [2], [3], learned regression [4], [5], or purpose-built staff-gauge and stage-camera systems [6], [7]. They need surveyed control points, dedicated cameras, or training labels from the very gauge one would want to check. We found none that uses the camera to *audit* the gauge.

Contributions.

1) A gauge-free reader for existing staff-plate cameras: the plate's own marks give a 1-D projective pixel-to-feet map, the printed foot numbers (read with `cv.dnn.TextRecognitionModel`) give the datum, and per-mark color contrast gives the waterline. It reads a flood higher than any water level seen in setup at 0.26 ft RMSE, and it returns a status instead of a guess when it cannot see.
2) An event-gated agent that arbitrates between two instruments: code detects disagreement, silence, blindness and flood levels; only then does Claude gather evidence (earlier frames, gauge history, an upstream gauge) and act. A rule planner with the same tools is the baseline.
3) A safe operating design, deployed serverless on AWS Graviton: agent memory, a safety net that always surfaces flood alerts, a rule fallback and a human approval queue, evaluated with repeated replays of injected gauge and camera faults on a real flood, failures included.

II. Related Work

Eltner et al. [2] segment water with a CNN and intersect the waterline with a photogrammetric 3-D model; Blanch et al. [3] operate the same family for years at dedicated stations with surveyed ground control. Vandaele et al. [4] transfer-learn water segmentation on public river cameras but stop at segmentation. Xu and Wang [5] assess deep networks that regress gauge height from webcam images; such models learn from the gauge record and are bounded by its range. Staff-gauge vision systems [6] and low-cost stage cameras [7] read water level at sites built for the purpose. StageSight reads an absolute stage from the plate of an existing public camera, needs neither survey nor training labels, and is designed to disagree with the gauge when the gauge is wrong.

III. Data

Camera. The NIMS "Hancock Staff" camera at USGS 01421500, 1080×1920 frames, one per hour, 21 July to 19 October 2026 (1,869 frames). The camera started on 14 April 2026; the 29 July event is its only flood. **Gauge.** USGS gage height (parameter 00065, 15-min), used only to evaluate the reader and, by the agent, as the instrument under test. **Upstream.** USGS 01417500 at Harvard, NY (an agent tool; cross-correlation over 90 days gives a lag of about 3 h). **Flood levels.** NWS NWPS, HKCN6: action 14 ft, minor flood 18 ft. All data are public.

IV. Reading the Plate with OpenCV 5

A. Setup, once per camera

Plate and marks. Red pixels are found by an HSV threshold and a 3×3 opening; `cv.fitLine` with a Huber loss gives the plate axis **d** through origin **o**. Each connected red component

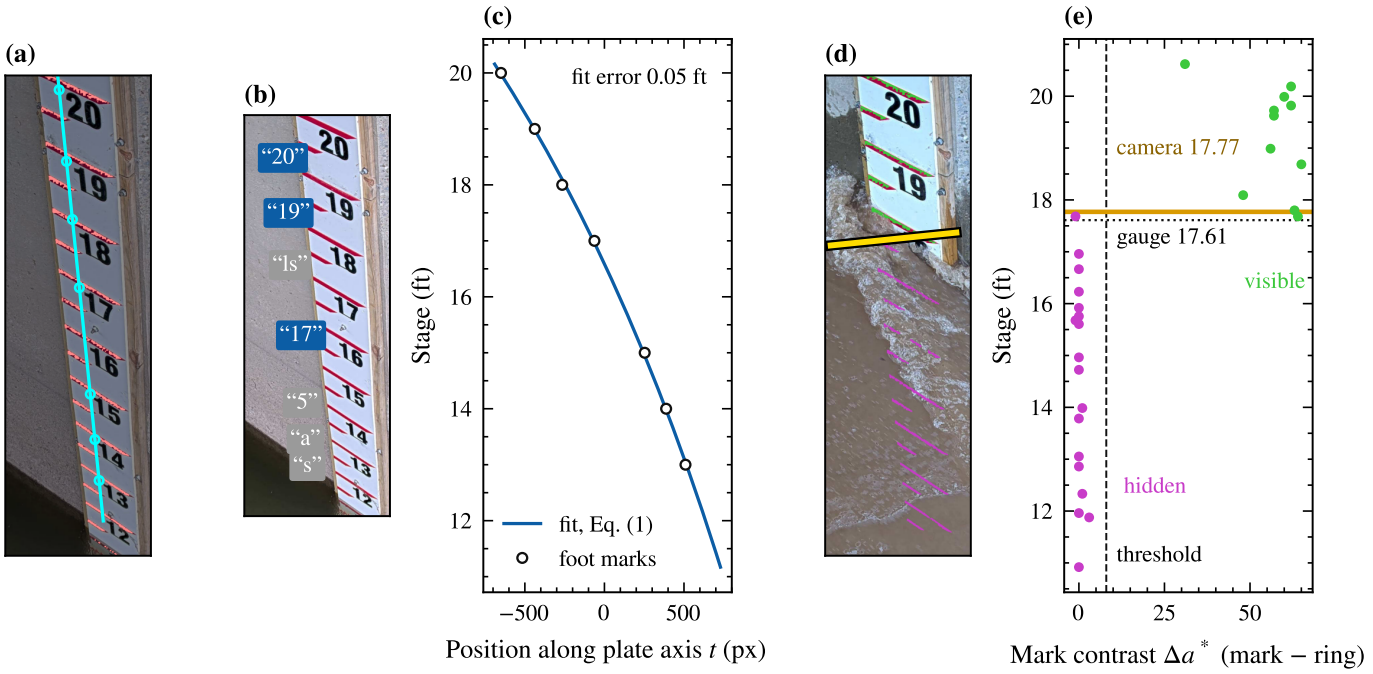


Fig. 1. The reader on real frames. Setup on the reference frame, no gauge data: (a) HSV red mask and robust plate axis (`cv.fitLine`, Huber) through the foot-mark centroids (`cv.connectedComponentsWithStats`); (b) CRNN text recognition of the printed numbers; two-digit reads (blue) vote for the datum, shorter misreads (grey) cannot; (c) the 1-D projective map from the foot marks, Eq. (1). Every frame, here near the second flood peak (29 July, 16:00 UTC): (d) marks still visible (green) and hidden under water (magenta), waterline in yellow; (e) the contrast of each mark against its surround, Eq. (2), on the stage axis: visible marks lie far above the threshold, submerged marks at zero.

is a mark. The long foot marks (area $\geq 0.6\times$ the median of the eight largest) are projected onto the axis, $t_i = (\mathbf{c}_i - \mathbf{o})^\top \mathbf{d}$; a gap of about twice the local spacing is indexed as a missing mark.

Scale. A pinhole camera maps points on a straight plate to the image line by a 1-D homography, so plate units u (ft below the top foot mark) follow

$$u(t) = \frac{a\,t + b}{c\,t + 1}, \qquad \begin{bmatrix} t_i & 1 & -t_i u_i \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = u_i, \tag{1}$$

solved by linear least squares over the foot marks (fit error 0.05 ft). Three parameters fixed by geometry extrapolate along the plate where a free polynomial would not.

Datum. The plate is rotated upright (`cv.warpAffine`), red is whitened, and the black digits beside each foot mark are cropped. The OpenCV Zoo CRNN [8], [9] runs through `cv.dnn.TextRecognitionModel` with CTC greedy decoding. In a frame, each read k at mark i proposes the datum $D = k + u_i$; a dropped leading "1" is the usual misread, so only the longest reads may vote. Across the setup frames the majority wins: $D = 20$ ft in 6 of 7 frames (`sites/hancock/site.json`). Stage is then $s = D - u$.

B. Every frame

Quality gate. Frames that are blurred (variance of the Laplacian below 40), dark or over-exposed are `rejected`; frames of another size are `wrong_size`. **Registration.** The camera moves a few pixels between days. `cv.phaseCorrelate` on the always-dry top of the plate (Hann-windowed) gives the shift; above 80 px the status is `camera_moved`. **Mark visibility.** For each of the $n = 29$ red components (foot and half-foot marks and their fragments), the median Lab a^* inside it is compared with a ring of non-red pixels around it (`cv.dilate`, 15×15 minus 5×5):

$$v_j = \mathbf{1}\left[\mathrm{med}_{\mathrm{mark}\ j}\, a^* - \mathrm{med}_{\mathrm{ring}\ j}\, a^* \geq 8\right]. \tag{2}$$

The test is relative, so it holds in sun, shade and overcast; water and foam over a mark remove its contrast. **Waterline.** With marks ordered top to bottom, the split

$$k^\star = \arg\min_k \ \textstyle\sum_{j<k}(1 - v_j) + \sum_{j\geq k} v_j \tag{3}$$

tolerates a few wrong marks (glare, a leaf). The waterline $t^\star$ is the midpoint of the gap between mark $k^\star - 1$ and mark $k^\star$, and $s = D - u(t^\star)$. Consecutive possible waterlines are 0.08–0.79 ft apart, which sets the resolution of one reading. **Status.** $k^\star \geq n - 1$ (all marks visible, or only the lowest hidden, which is ambiguous at low water) gives `below_plate`; $k^\star = 0$ gives `unreadable` (night infrared, fog, spray). The reader never extrapolates below the plate.

V. The Agent

OpenCV runs on every frame; Claude is called only when code raises an event (Table I). The planner receives the

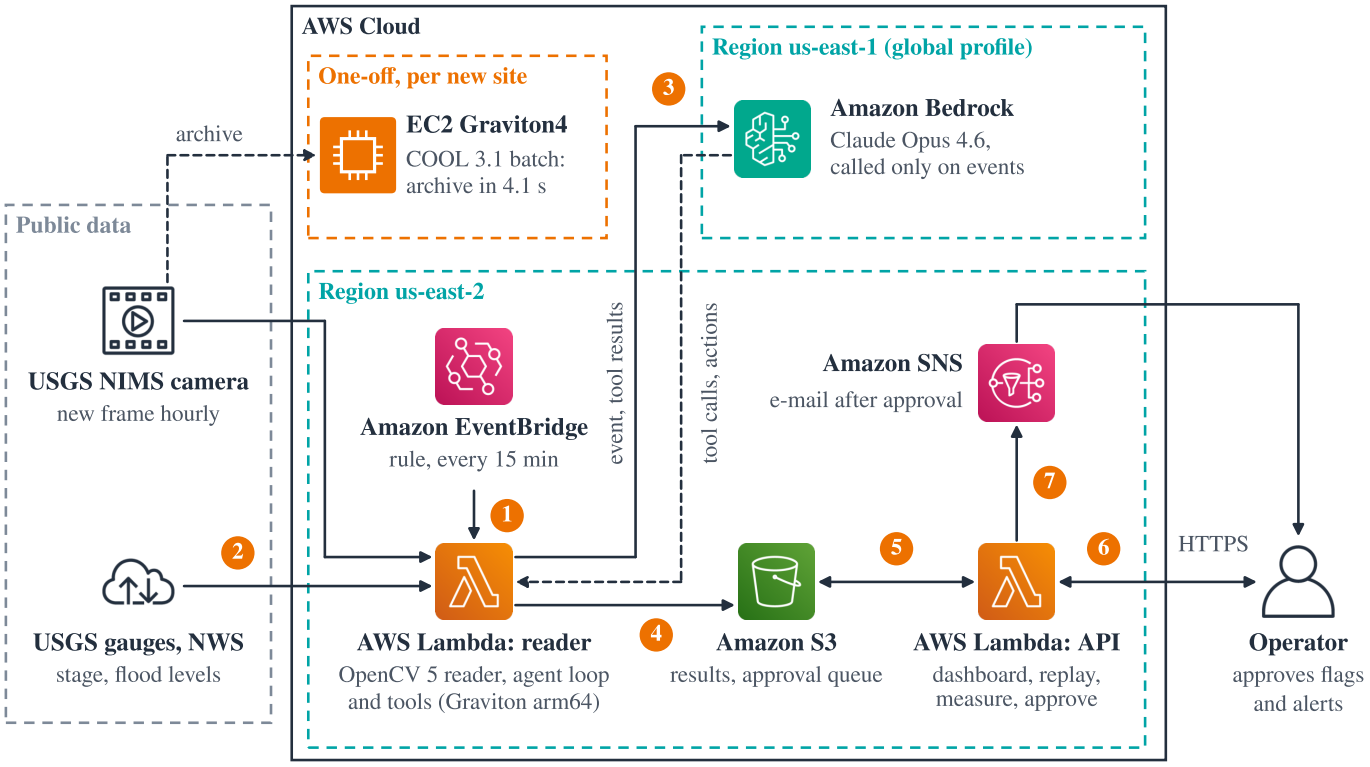


Fig. 2. Deployment. ① An EventBridge rule starts the reader Lambda every 15 min; ② it fetches the newest frame (one per hour, each processed once) and gauge values; ③ on an event it runs the agent loop, in which Claude on Bedrock chooses tools and the Lambda executes them; ④ results and the approval queue go to a private S3 bucket; ⑤–⑥ a second Lambda serves the dashboard, the flood replay and the measure endpoint, and records the operator's decisions; ⑦ an approved flag or alert is e-mailed through Amazon SNS, the only path out of the system. Top left: the one-off batch that re-reads a new site's camera archive on EC2 Graviton4 with COOL (Sec. VI).

TABLE I
EVENTS THAT CALL THE AGENT (CHECKED BY CODE ON EVERY FRAME)

Event	Trigger
`disagree`	camera and gauge differ by > 0.5 ft for 2 frames, or the camera is below the plate while the gauge reads > 0.5 ft above it
`gauge_silent`	the camera reads, but the gauge has no value in the last 30 min (90 min live, for the USGS reporting delay)
`camera_blind`	camera unreadable or moved for 3 frames while the gauge is at or above action stage
`level_rise`	the stage (camera when readable, else gauge) reaches an NWS level

event, the open issues and its own recent actions (memory), and runs a manual tool loop with adaptive thinking through the Anthropic Bedrock client. Evidence tools: `reread_earlier_frames` (re-runs the reader, with mark counts), `get_site_gauge`, `get_upstream_gauge`. Final actions: `flag_sensor` and `draft_alert` (both enter the approval queue), `publish_provisional_stage` and `no_action`. Two operator facts are in the prompt: readings move in steps of about 0.5 ft, and upstream changes arrive about 3 h later. The prompt also gives the reader's accuracy, 0.26 ft RMSE, which was measured on the same flood.

Safety design. (i) *Safety net*: a `level_rise` always yields an alert draft for the operator; if the agent doubted it, its doubt is written into the draft. (ii) *Fallback*: on any API or connection error, a refusal, or no final action, the rule planner decides and the log says so. (iii) *Human in the loop*: no flag or alert leaves the system without an operator's approval.

VI. System on AWS

Both functions run on arm64 (Graviton) from one zip package that bundles the Linux arm64 wheels of `opencv-python-headless` 5.0.0.93 and the flood replay; it is built without Docker and deployed by an idempotent script that creates the bucket, the IAM role, both functions, the schedule, a Function URL and an SNS topic for approved alerts (Fig. 2). The role can write logs, read and write only the project bucket, invoke Anthropic models on Bedrock and publish to that one topic. From CloudWatch, a scheduled tick (download, read, event check, write) takes a median of 6.3 s ($n = 23$, peak memory 295 MB) and an API request a median of 47 ms ($n = 1{,}227$). Claude is called only on events: none on a quiet day, two or three during the flood, each with three evidence calls and one to three actions. With Opus 4.6 one replayed scenario used about 29k input and 5k output tokens per scenario (median of three runs; adaptive thinking, effort medium).

TABLE II
COOL VS PIP OPENCV ON THE SAME GRAVITON4 INSTANCES

	COOL 3.1	pip 5.0.0	ratio
c8g.xlarge (4 vCPU), one process, 184 flood frames			
reader, frames/s	59.2	56.9	1.04×
`connectedComponentsWithStats`	0.40 ms	1.97 ms	4.90×
`phaseCorrelate` (plate crop)	3.79 ms	4.75 ms	1.25×
`morphologyEx` (opening)	0.41 ms	0.55 ms	1.32×
`imdecode` (1080×1920 JPEG)	7.79 ms	6.97 ms	0.90×
DNN text recognition (datum)	crashed	0.25 s	–
c8g.4xlarge (16 vCPU)			
reader, one process, frames/s	71.6	68.8	1.04×
archive batch, 16 workers, frames/s	454	464	0.98×
DNN text recognition (datum)	0.19 s	0.18 s	–

Medians; kernels over 50 calls. Identical stages from both builds in every run. Prices: AWS Pricing API, us-east-2 on-demand, plus the COOL software fee. `results/cool/`.

TABLE III
STAGE ERROR ON THE 29 JULY 2026 FLOOD (n = 30)

Method	Gauge data	RMSE	Bias	≤0.5 ft
StageSight (ours)	none	**0.26**	+0.07	**97%**
Plate + gauge-fitted offset	training	0.37	−0.27	83%
Linear pixel fit	training	6.31	−4.70	23%

Errors in ft. StageSight MAE 0.21 ft, R^2 = 0.99; RMSE 95% bootstrap interval 0.20–0.31 ft.

COOL on Graviton4. The live tick runs on Lambda; a new site needs a batch pass over its camera archive, and that pass runs on EC2 Graviton4 with the Cloud-Optimized OpenCV Library (COOL 3.1: OpenCV 5.1-dev with KleidiCV 0.7.0, built for Neoverse-V2). We measured it against the pip `opencv-python-headless` 5.0.0 that Lambda uses, on the same instance (Table II; `scripts/cool_run.py` launches the instance, runs both builds, uploads the results and terminates it). Both builds read the same status and stage in every frame. With 16 worker processes a c8g.4xlarge reads the whole 90-day archive (1,869 frames) in 4.1 s, about $0.0008 at on-demand prices ($0.0004 per 1,000 frames); throughput scales almost linearly from 33 frames/s with one worker. COOL is 4% faster in a single process and up to 4.9× faster in single kernels, but the batch is bound by NumPy and Python, so both builds tie there. COOL's DNN text recognizer aborted on c8g.xlarge in both recorded runs (no OCR result in either) and ran normally on c8g.4xlarge; a minimal reproduction is in `scripts/cool_dnn_repro.py`.

VII. EXPERIMENTS

A. Accuracy on a real flood

Test frames are the daylight frames of 21 July–1 August with the water on the plate (n = 30, 29–31 July). The baselines are trained on the 15 frames of the 4–5 September high water (up to 11.97 ft); 27 of the 30 test frames lie above that range.

As Table III and Fig. 3 show, StageSight is the only gauge-free method and the most accurate. Fitting the offset to the gauge is worse (0.37 ft): the lowest mark is often wet at low water, which biases the fit. The linear baseline fails because the flood lies outside its training range. Two flood frames are regression tests in the repository (17.77 vs 17.61 ft; 12.63 vs 12.47 ft). The 30 test frames are hourly and correlated, so the bootstrap interval is optimistic; they come from one flood at one camera.

Second event. The reader never used the 15 daylight frames of the 4–5 September high water (gauge 11.63–11.97 ft); only the baselines were trained on them. Scored as a test event (OCR datum from frames outside it, 9 of 11 agree on 20 ft), StageSight has 0.34 ft RMSE (95% bootstrap 0.29–0.38 ft), bias +0.33 ft, 15 of 15 within 0.5 ft. The water stayed inside one gap between two marks, so all 15 readings are the gap midpoint, 12.11 ft: one measurement repeated, whose error is set by the mark spacing (`results/hancock_plate_sep/`).

Ablations. A sub-mark edge search inside the bracket gave 0.275 ft RMSE against 0.257 ft for the midpoint and was not adopted. Using the bracket between two marks as an uncertainty interval covered the gauge in only 23–50% of frames, so we report no intervals.

B. Coverage

Over the 1,863 hourly frames of the 90-day replay (21 July to 18 October) the reader gave a number for 45 (2.4%, Fig. 4); the water was below the plate for 64% and 33% were night or obscured. The plate is a flood instrument: it reads when the river is high, which is when a gauge fault matters most.

Real degraded frames. We searched the archive for daylight frames with the water on the plate (gauge ≥11 ft, 87 frames) spoiled by rain on the lens, fog or glare. There are none: the quality gate rejected no frame and none looks degraded by eye. Over all 1,249 daylight frames no frame was rejected or `unreadable`, so the obscured share above is night infrared only. The one real failure is hydraulic: at 29 July 15:00 UTC whitewater ran up the plate and the reader gave 17.77 ft against 17.14 ft (+0.63 ft), the only July test frame outside 0.5 ft (`results/hancock_real_faults/`).

C. Agent: injected faults on the real flood

The replay feeds the 29 July flood frame by frame and injects four gauge faults and two camera faults; the camera faults are synthetic overlays on real frames, placed by us. A fault counts as caught when the gauge is flagged (or, for a silent gauge, the camera stage is published) within the fault window plus 2 h; a camera fault counts as handled when the camera is blamed and the healthy gauge is not. The rule planner uses the same tools with fixed thresholds. Each Claude model ran every scenario 3 times (Table IV).

Opus 4.6 caught every fault in all three runs with the same delays; it flagged the frozen gauge after 2 h against 4 h for the rules (Fig. 5) and the drift 1 h earlier. It was not error-free: in the frozen-gauge case it also flagged the healthy camera at the second peak in all three runs (Sec. VII-D), and in that case the flood alert came from the safety net, not from Claude. Sonnet 4.6 made the same camera error and, on the slow drift, blamed the camera instead of the gauge in two of three runs, once before the fault began. In one Sonnet run

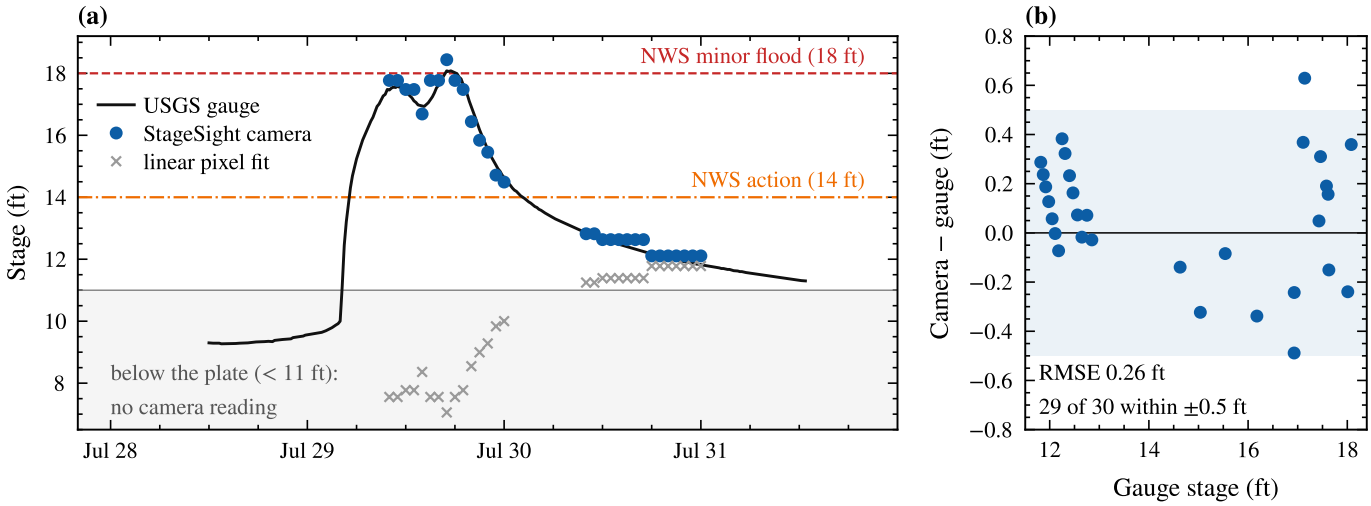


Fig. 3. (a) The camera follows both flood peaks with no gauge data; a linear fit trained on the 4–5 September high water cannot reach flood stage. Gaps in the camera series are nights. Below the plate (shaded) the reader reports `below_plate`. (b) Error against stage; the band is ±0.5 ft. Runs of equal readings near 12–13 ft are the step between two marks.

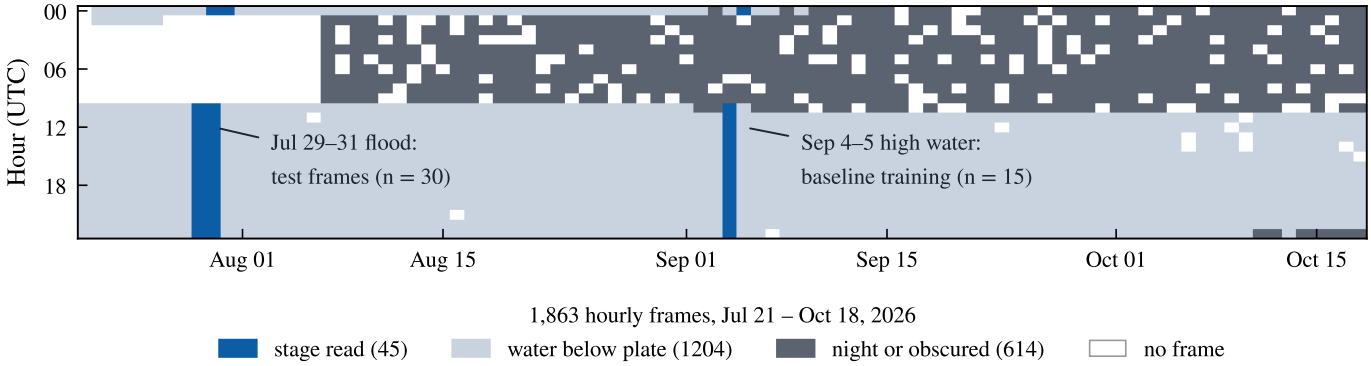


Fig. 4. Reader status for every hourly frame of the 90-day replay (columns: days; rows: hour, UTC). The camera reads during the two high-water periods; otherwise the water is below the plate (light) or it is night (dark).

TABLE IV
INJECTED FAULTS ON THE 29 JULY FLOOD: RUNS THAT CAUGHT THE FAULT AND MEDIAN DELAY (RANGE)

Fault	Rules	Opus 4.6	Sonnet 4.6
gauge offset +1 ft	1/1, 1 h	3/3, 1 h	3/3, 1 h
gauge silent 8 h	1/1, 1 h	3/3, 1 h	3/3, 1 h
gauge frozen 10 h	1/1, 4 h	3/3, 2 h	3/3, 2 h
gauge drift to +1.5 ft	1/1, 8 h	3/3, 7 h	1/3, 7 h
debris over plate	0/1	3/3	3/3
spray on lens	0/1	3/3	3/3
flood alert in every scenario	1/1	3/3	3/3
false sensor flags (all runs)	0	0	1
healthy camera also flagged	0	3	5
alerts drafted by the safety net	0	4	3

Rules are deterministic (one run); each Claude model ran every scenario 3 times. Delay: hours from the fault start to the gauge flag (silent gauge: to the published camera stage). Camera faults: the camera blamed, the healthy gauge not blamed, the bad camera stage never published. Last two rows: counts over all runs and gauge-fault scenarios.

Bedrock answered once with `ServiceUnavailable`; the rule planner decided that event, as designed.

Fixed thresholds cannot tell which instrument is wrong: in both camera faults they see a persistent disagreement, blame the gauge and publish the bad camera stage (Fig. 6). Claude checks the evidence and blames the camera (Fig. 7). The rule planner was not given the mark-count signal Claude used; a rule written for it is an obvious extension that we did not test.

In 90 days of normal operation (1,863 frames) neither the rule planner nor Claude Opus 4.6 (one run, 17k input and 2.9k output tokens) raised a false sensor flag; for both, the two alerts were an action alert and a flood alert, both at the real flood. Only two events reached the planner in those 90 days, both during the flood, so this is a small sample, not a measured false-alarm rate.

D. Development history of the agent

We report every Claude run. *Run 1* (no saved log) missed the drift and blamed the camera; the two operator facts above were added. *Run 2* caught every fault but, in the frozen-gauge case, forgot its own gauge flag and drafted no flood alert during a real flood. The fix was in code, not in the prompt: memory

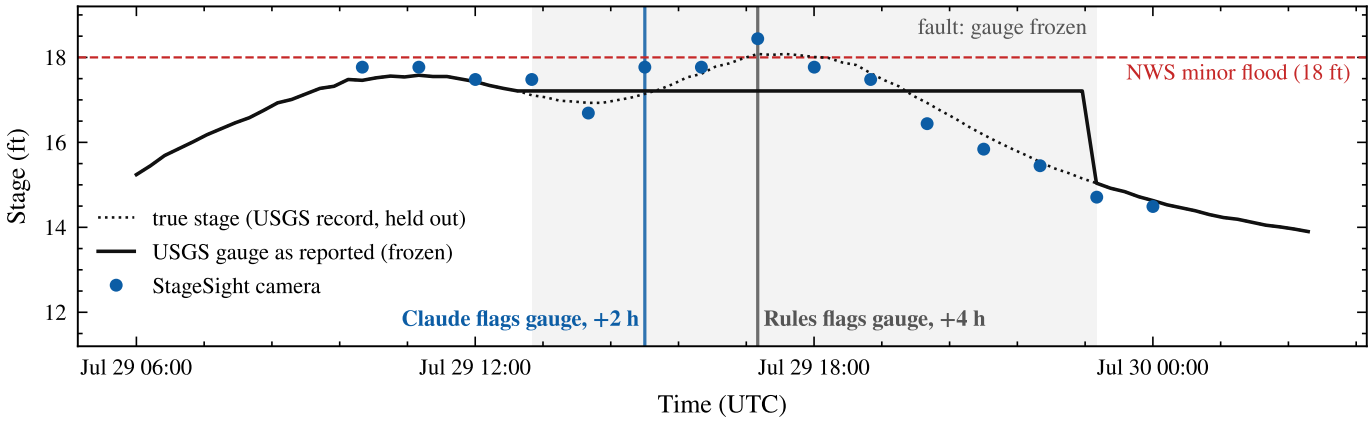


Fig. 5. Frozen-gauge replay (one Opus 4.6 run). The reported gauge stays at 17.21 ft while the river rises to its second peak and falls. Claude flags the gauge 2 h after the fault starts (13:00 UTC); the rule baseline flags it after 4 h, once three frames disagree consistently.

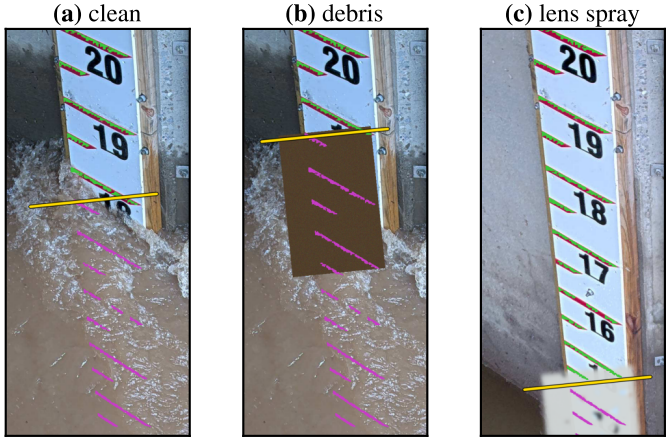


Fig. 6. Simulated camera faults on real frames (29 July 18:00 and 30 July 10:00 UTC) with the reader's output and the agent's verdict. (b) Debris hides the marks from 16.6 to 18.8 ft; (c) spray removes their color. The reader reads too high and the gauge is right.

of open issues and the safety net. *Run 3*, the single run shown in Fig. 5: the safety net fired once, again in the frozen-gauge case. Claude flagged the frozen gauge, but partly for a wrong reason: it inferred from the 3 h lag that the river was receding, while this flood peaked about 7 h after the upstream peak. It therefore dismissed the 18.44 ft flood-level camera reading at 17:00 (true 18.08 ft) and flagged the camera, and its provisional stages were 0.3–0.45 ft low. We did not re-tune the lag to this flood. The repeated runs in Table IV use the code of run 3.

VIII. Limitations and Threats to Validity

One flood, one camera. The accuracy rests on 30 correlated frames of one event, and the agent results on replays of that event; the second event (15 frames) stays inside one mark gap. The camera faults are synthetic, because the archive holds

Claude Opus 4.6, debris case, 29 July 19:00 UTC (verbatim)

Tools: `reread_earlier_frames(n=4), get_site_gauge(12 h), get_upstream_gauge(12 h).`
flag_sensor(camera), evidence: "Camera reads 18.74 ft vs gauge 17.63 ft (+1.11 ft, >4× RMSE). Visible marks fell from 11 to 7 even as the gauge recedes smoothly from its 18.06 ft peak. Upstream gauge confirms falling trend with appropriate 3-hour lag. [...]" (The lag Claude cites is the prompt's 3 h; for this flood it was about 7 h, Sec. VII-D.)

Fig. 7. An agent decision in the debris case: the evidence Claude gathered and the flag it queued for approval.

no real rain, fog or glare frame. **Ground truth.** The USGS gauge is the reference, although it is the instrument the system questions. **Night**: infrared frames show the plate but not the red marks (`unreadable`). **Low water**: below about 11 ft the plate is dry and the camera cannot check the gauge. **Plate designs**: other NIMS plates (Owasco, Moravia, Chenango, NY) use black marks or 0.1 ft text, or stand on dry bank; they are not supported yet. **No plate**: a self-calibrated research mode learns a per-pixel flood-stage map from the archive (0.29 ft MAE in range on the Hancock wall) but does not extrapolate to floods; on far-bank bridge cameras (Hermann, MO; De Soto, KS; 2–3 px per ft) it reaches only 3.4 and 2.3 ft RMSE in range. **Models**: Claude Opus 4.6 and Sonnet 4.6 are the Claude models available on this AWS account; the model is a configuration setting (`AGENT_MODEL`).

IX. Responsible Operation

StageSight is a decision aid, not an official warning system. Nothing leaves the system until an operator approves it; approved flags and alerts are then e-mailed through Amazon SNS. Live approvals require an operator token, and approvals in the public replay change nothing. A camera stage that stands in for the gauge is labeled provisional. The system uses public data and no personal data. The bucket is private, the role is limited as above, failed ticks are not retried, and every agent decision, tool call and reason is logged and shown on the dashboard.

The public `/measure` endpoint accepts only frames of this camera's size (1080×1920).

X. Conclusion

A camera that already watches a staff plate can measure stage by geometry and OCR alone, through a flood above everything seen in setup, and that independent reading lets an agent tell which instrument is wrong. Next: read night infrared frames from the printed numbers and support more plate designs across the NIMS network.

Reproducibility. Code, results and figures: https://github.com/Waqar-msx9t/stagesight (Python 3.12, OpenCV 5.0.0.93, NumPy 2.5.4, Anthropic SDK 1.15.0). The tests run offline; the scripts *plate_test*, *replay_agent* and *make_figures* reproduce every number and figure.

References

[1] U.S. Geological Survey, "National Imagery Management System (NIMS) API," https://api.waterdata.usgs.gov/nims/v0, accessed Oct. 2026.

[2] A. Eltner, P. O. Bressan, T. Akiyama, W. N. Gonçalves, and J. Marcato Junior, "Using deep learning for automatic water stage measurements," *Water Resources Research*, 2021, doi:10.1029/2020WR027608.

[3] X. Blanch, J. Grundmann, R. Hedel, and A. Eltner, "AI image-based method for a robust automatic real-time water level monitoring: a long-term application case," *Hydrology and Earth System Sciences*, vol. 30, p. 797, 2026, doi:10.5194/hess-30-797-2026.

[4] R. Vandaele, S. L. Dance, and V. Ojha, "Deep learning for automated river-level monitoring through river-camera images: an approach based on water segmentation and transfer learning," *Hydrology and Earth System Sciences*, vol. 25, p. 4435, 2021, doi:10.5194/hess-25-4435-2021.

[5] R. Xu and B. Wang, "Assessing deep learning techniques for remote gauging and water quality monitoring using webcam images," *Hydrology*, vol. 12, no. 4, p. 65, 2025, doi:10.3390/hydrology12040065.

[6] L. Sabbatini, L. Palma, A. Belli, F. Sini, and P. Pierleoni, "A computer vision system for staff gauge in river flood monitoring," *Inventions*, vol. 6, no. 4, p. 79, 2021, doi:10.3390/inventions6040079.

[7] S. Noto, F. Tauro, A. Petroselli, C. Apollonio, G. Botter, and S. Grimaldi, "Low-cost stage-camera system for continuous water-level monitoring in ephemeral streams," *Hydrological Sciences Journal*, 2022, doi:10.1080/02626667.2022.2079415.

[8] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," arXiv:1507.05717, 2015.

[9] OpenCV, "OpenCV Zoo: text recognition CRNN (text_recognition_CRNN_EN_2021sep)," https://github.com/opencv/opencv_zoo.